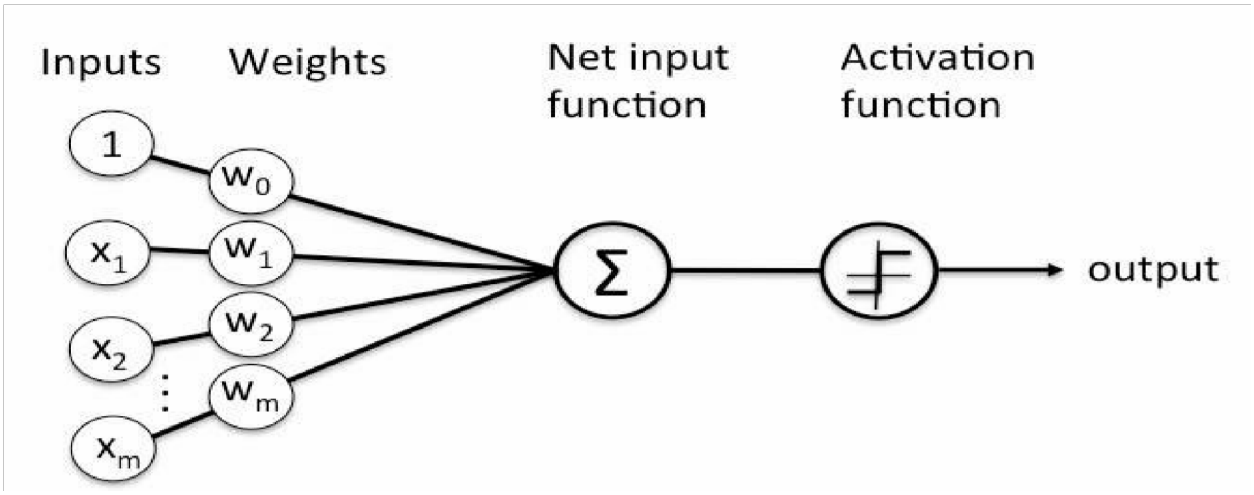
Table of Contents

1 Single Perceptron

A perceptron is a neural network unit (an artificial neuron) that does certain computations to detect features or business intelligence in the input data. A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and process elements in the training set one at a time.



1.1 Types of Perceptrons

1) **Single layer Perceptrons** can learn only linearly separable patterns.
2) **Multilayer Perceptrons** or feedforward neural networks with two or more layers have the greater processing power.

1.2 Perceptron Function

Perceptron is a function that maps its input "x," which is multiplied with the learned weight coefficient; an output value "f(x)"is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

In the equation given above:

"w" = vector of real-valued weights

"b" = bias (an element that adjusts the boundary away from origin without any dependence on the input value)

"x" = vector of input x values

$$\sum_{i=1}^{m} w_i x_i$$

"m" = number of inputs to the Perceptron

The output can be represented as "1" or "0." It can also be represented as "1" or "-1" depending on which activation function is used.

1.3 Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.

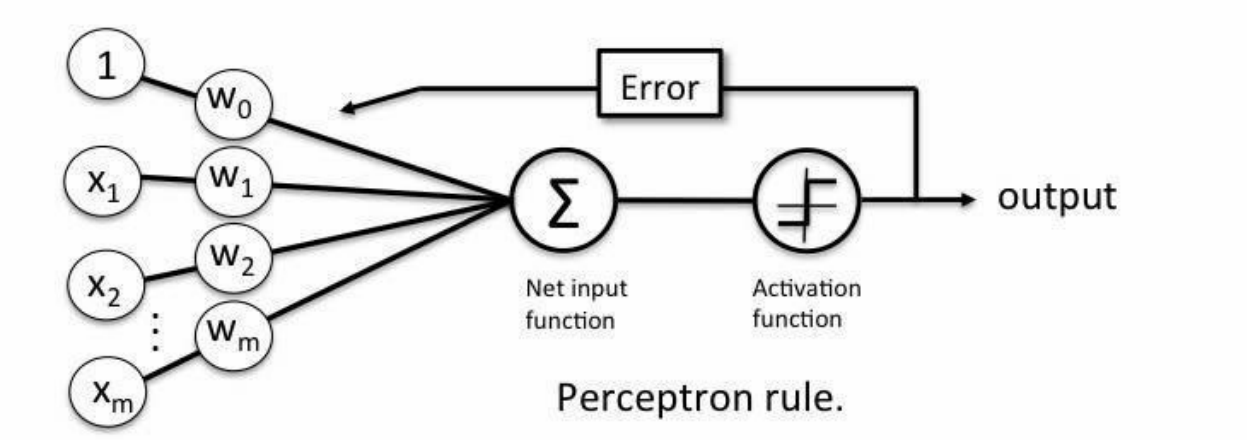


Figure 2 Inputs in a Perceptron Learning Process

A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function "∑" multiplies all inputs of "x" by weights "w" and then adds them up as follows:

$$w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

1.4 Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.

For example:

If ∑ wixi> 0 => then final output "o" = 1 (issue bank loan)

Else, final output "o" = -1 (deny bank loan)

Step function gets triggered above a certain value of the neuron output; else it outputs zero.

1.5 Perceptron at a glance

- Weights are multiplied with the input features and decision is made if the neuron is fired or not.
- Activation function applies a step rule to check if the output of the weighting function is greater than zero.
- Linear decision boundary is drawn enabling the distinction between the two linearly separable classes +1 and -1.
- If the sum of the input signals exceeds a certain threshold, it outputs a signal; otherwise, there is no output.
- Types of activation functions include the sign, step, and sigmoid functions.

1.6 Perceptron Training Algorithms

In this course, we have covered training algorithms namely, perceptron learning.

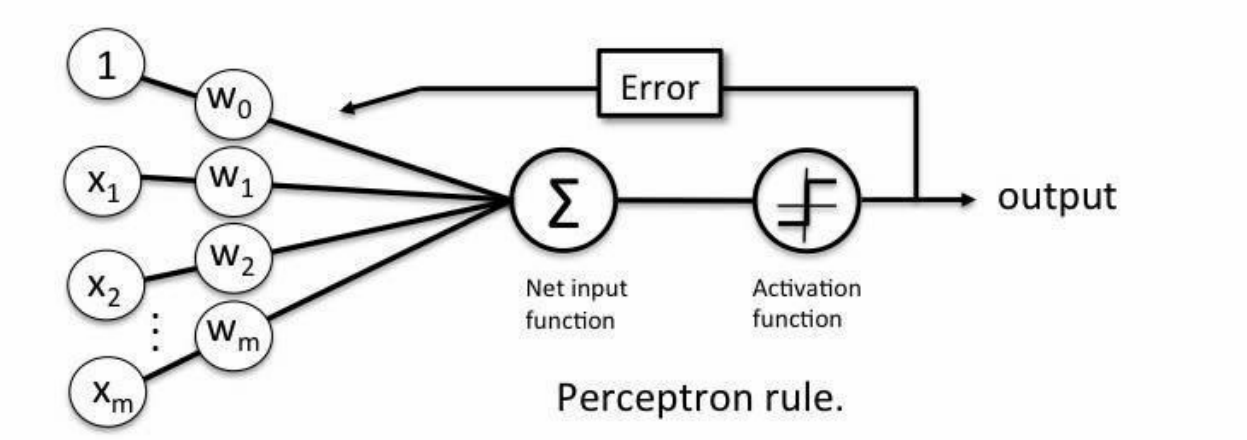


Figure 4 Learning Process

The perceptron receives multiple input signals and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output.

In this lab session, we only focus on the perceptron learning rule. This rule requires the data to be linearly separable. The weight update rule is given by:

$$w_i = w_i + \Delta w_i$$

where

$$\Delta w_i = \eta \cdot (y - \hat{y}) \cdot x_i$$

and

$$b = b + \eta \cdot (y - \hat{y})$$

Where:

- y is the actual output,
- $\hat{y}$ is the perceptron's predicted output,
- η is the learning rate (a small constant that controls how much weights change).

1.7 Logic Gates

Logic gates are the building blocks of a digital system, especially neural networks. In short, they are the electronic circuits that help in addition, choice, negation, and combination to form complex circuits.

Using the logic gates, Neural Networks can learn on their own without you having to manually code the logic. Most logic gates have two inputs and one output.

Each terminal has one of the two binary conditions, low (0) or high (1), represented by different voltage levels. The logic state of a terminal changes based on how the circuit processes data. Based on this logic, logic gates can be categorized into six types:

1) AND
2) NAND
3) OR
4) NOR
5) NOT
6) XOR

2 Introduction to tensorflow and pytorch

Modern Artificial Intelligence (AI) and Machine Learning (ML) applications rely heavily on *deep learning* frameworks that simplify the implementation and training of complex neural networks. Two of the most widely used open-source libraries for this purpose are **TensorFlow** and **PyTorch**. Both are built for **numerical computation using dataflow graphs** and provide powerful APIs for building Artificial Neural Networks (ANNs) efficiently.

2.1 What is TensorFlow?

TensorFlow is an open-source platform for machine learning and deep learning developed by Google. It allows researchers and developers to build and train neural networks using multi-dimensional arrays called **tensors**.

Key Features of TensorFlow

- **Tensors:** Core data structure; represents data as n-dimensional arrays.
- **Computation Graphs:** Operations are represented as nodes in a graph (static or eager execution mode).
- **Keras API:** High-level interface in TensorFlow that simplifies model creation, training, and evaluation.
- **GPU/TPU Support:** TensorFlow can leverage hardware acceleration for faster training.
- **Model Deployment:** Integration with TensorFlow Lite, TensorFlow.js, and TensorFlow Serving allows models to be deployed on mobile, web, and production servers.

2.2 What is PyTorch?

PyTorch is an open-source deep learning framework developed by Facebook (Meta) that provides a flexible and dynamic way to define and train neural networks. Unlike TensorFlow's early versions, which used static computation graphs, PyTorch uses **dynamic computation graphs**, allowing changes in network structure during runtime, very useful for research and experimentation.

Key Features of PyTorch

- **Dynamic Computation Graphs:** Build and modify networks on the fly during execution.
- **Pythonic Syntax:** Seamless integration with Python libraries such as NumPy.
- **Automatic Differentiation:** The `autograd` module computes gradients automatically for backpropagation.
- **TorchScript & Deployment:** Models can be exported for production with TorchScript or deployed via ONNX.
- **GPU Support:** Built-in CUDA support for efficient GPU computations.

3 Single-Layer Perceptron using TensorFlow

3.1 Import necessary libraries

```
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

3.2 Create and split synthetic dataset

We will create a simple 2-feature synthetic binary-classification dataset for our demonstration and then split it into training and testing.

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_repeated=0,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

3.3 Standardize the Dataset

Now standardize the dataset to enable faster and more precise computations. Standardization helps the model converge more quickly and often enhances accuracy.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3.4 Building a neural network

Next, we build the single-layer model using a Sequential architecture with one Dense layer. The Dense(1) indicates that this layer contains a single neuron. We apply the sigmoid activation function, which maps the output to a value between 0 and 1, ideal for binary classification tasks. The original perceptron used a step function that produced only 0 or 1 as outputs and trained

differently, but modern models use the sigmoid function because it's smooth and enables learning through gradient-based optimization. We define the input shape separately using tf.keras.Input(shape=(2,)), which specifies that each input sample consists of two features. This approach follows the recommended Keras practice for clarity and compatibility.

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
```

Common activation Options in tf.keras

Activation functions introduce non-linearity into neural networks, enabling them to learn complex patterns. They are applied to each layer's output to control what gets passed to the next layer.

Activation	Description	Typical Use Case
linear	Returns the input as-is: *f(x) = x*.	Regression outputs or identity mapping
sigmoid	Maps values between 0 and 1.	Binary classification output
softmax	Converts logits into probabilities that sum to 1.	Multiclass classification output
tanh	`Maps values between -1 and 1.`	Hidden layers when data is centered around zero
relu	Rectified Linear Unit: *f(x) = max(0, x)*.	Most common activation for hidden layers

Example:

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(2,)),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

3.5 Compiling the model

Next, we compile the model to prepare it for training. The compile() method defines how the model will learn and be evaluated. We use the Adam optimizer, a popular gradient-based optimization algorithm that adapts the learning rate during training for efficient convergence. The binary_crossentropy loss function is chosen because this is a binary classification problem, it measures the difference between predicted probabilities and actual class labels (0 or 1). Finally, we include accuracy as a metric to monitor how often the model's predictions match the true labels during training and evaluation.

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Common optimizer Options in tf.keras

Optimizers control how the model updates its weights during training. Some common optimizers are:

Optimizer	Description
SGD	Stochastic Gradient Descent. A simple and standard optimizer. It can use *momentum* to speed up convergence.
RMSprop	Often used for recurrent neural networks. Adapts learning rates for each parameter.
Adam	A popular default optimizer. Combines the benefits of RMSprop and momentum.
Adagrad	Adapts the learning rate based on how frequently parameters are updated. Good for sparse data.
Adadelta	Similar to Adagrad but avoids indefinitely accumulating past squared gradients.
Nadam	Adam optimizer combined with Nesterov momentum.

Common loss Options in tf.keras

A loss function measures how well the model predicts the target values.

1. For Classification

Loss	Use Case
binary_crossentropy	Used for binary classification problems (two classes such as 0 or 1).
categorical_crossentropy	Used for multi-class classification with one-hot encoded labels.
sparse_categorical_crossentropy	Used for multi-class classification when labels are integers instead of one-hot encoded.
hinge	Used for binary classification with a maximum-margin approach (similar to SVM).

2. For Regression

Loss	Use Case
mean_squared_error (MSE)	Standard regression loss that measures squared differences between predicted and actual values.
mean_absolute_error (MAE)	Measures the absolute difference between predicted and actual values; less sensitive to outliers.
mean_absolute_percentage_error (MAPE)	Measures prediction error as a percentage.
huber	A combination of MSE and MAE that is more robust to outliers.

You can also define a custom loss function by passing a Python function.

3.6 Train the Model

Now, we train the model by iterating over the entire training dataset a specified number of times, called epochs. During training, the data is divided into smaller batches of samples, known as the batch size, which determines how many samples are processed before updating the model's weights. We also set aside a fraction of the training data as validation data to monitor the model's performance on unseen data during training.

```
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=16,
                    validation_split=0.1)
```

3.7 Model Evaluation

After training we test the model's performance on unseen data.

```
loss, accuracy = model.evaluate(X_test, y_test)
```

If you want to add other evaluation metrics, when you compile the model, you can list multiple metrics to track:

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy', 'Precision', 'Recall', 'AUC'])
```

Now when you run:

```
results = model.evaluate(X_test, y_test, verbose=0)
print(results)
```

You will get a list like: [loss, accuracy, precision, recall, auc]

You can also unpack them:

```
loss, accuracy, precision, recall, auc = model.evaluate(X_test, y_test,
verbose=0)
```

If you want more detailed metrics such as confusion matrix, F1-score, or classification report use scikit-learn

```
from sklearn.metrics import classification_report, confusion_matrix

# Get predicted probabilities
y_pred_prob = model.predict(X_test)

# Convert probabilities to binary predictions (0 or 1)
y_pred = (y_pred_prob > 0.5).astype(int)

# Print detailed evaluation
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```